

DEVSECOPS & PLATFORM · ISSUE 07 OF 08

Logs, metrics and traces in practice.

Alert on symptoms. Debug with traces. Prove with logs.

07

AR

Akhilesh Ranjan Singh

Senior DevSecOps Engineer · github.com/iarsingh

Swipe →

Each pillar's job

SIGNAL	GOOD FOR	COST DRIVER
Metrics	Alerting, trends	Label cardinality
Logs	Detail, audit	Volume, retention
Traces	Cross-service latency	Sampling rate



Two checklists

RED — SERVICES

- Rate
- Errors
- Duration

USE — RESOURCES

- Utilization
- Saturation
- Errors



From alert to root cause

- 1 SLO alert
- 2 Service dashboard
- 3 Trace exemplar
- 4 Slow span
- 5 Logs with trace_id

The trace_id in every log line is what makes this work.



Keep it affordable

DO

- ✓ Structured JSON logs
- ✓ trace_id on every log line
- ✓ Sample traces, keep errors
- ✓ Tiered retention

AVOID

- ✗ user_id as a metric label
- ✗ Debug logs in prod forever
- ✗ Alerts on causes, not symptoms
- ✗ Dashboards nobody opens

Instrumentation baseline

- ☒ RED metrics per service
- ☒ OpenTelemetry trace propagation
- ☒ Structured logs with trace_id
- ☒ SLO dashboard
- ☒ Runbook link in every alert



KEY TAKEAWAY

Connect the signals with trace IDs, and control cardinality.

- Metrics alert, traces locate, logs explain
- RED for services, USE for resources
- Cardinality is the cost

YOUR TURN

How long does it take your team to go from alert to the right log line?

FOLLOW AKHILESH RANJAN SINGH

Next in the series: #08 An internal developer platform teams actually use

Save · Repost

